

Topic 5 - API Workflow Assessment

Building API Workflows in n8n

Implementation & Submission Documentation

This document describes the completed sequential API workflow, its dynamic expressions and execution-context mapping, the API calls, timestamp handling, and execution logging implemented for the Topic 5 assessment.

Workflow: Topic 5 - API Workflow Assessment

Workflow ID: PUclSNuyf1esa2zI

API: JSONPlaceholder REST API

Target User ID: 1

Assessment objective: demonstrate the ability to build multi-node API workflows, map data between nodes, use dynamic expressions and ancestor references, generate runtime headers and ISO-8601 timestamps, map execution/workflow context variables, and validate execution results.

1. Workflow Overview

The workflow is a sequential five-node chain. A manual start triggers request-context generation, followed by two live HTTP requests and a final Code node that combines the returned data with execution and workflow context.

#	Node	Purpose
1	Start	Manual trigger for the workflow.
2	Prepare Request Context	Creates requestTimestamp, requestId, and targetUserId.
3	Get User	Fetches the target user using a dynamic URL and runtime headers.
4	Get User Posts	Fetches posts using an ancestor-node reference and dynamic headers.
5	Build Result & Execution Log	Merges user, posts, request context, and execution metadata; writes a log line.

Flow: Start → Prepare Request Context → Get User → Get User Posts → Build Result & Execution Log

2. Requirement Coverage

Requirement	Implementation	Status
Multi-node sequential workflow	Five connected nodes with two sequential HTTP API calls.	PASS
\$json expressions	Get User uses \$json.targetUserId, \$json.requestId and \$json.requestTimestamp.	PASS
Ancestor node references	Get User Posts references Prepare Request Context; final Code node references Build Result & Execution Log.	PASS
Dynamic HTTP headers	Runtime request ID, timestamp, execution ID, workflow name, execution mode and ISO-8601 timestamp.	PASS
ISO-8601 timestamps	\$now.toISO() and new Date().toISOString() are used.	PASS
Execution/workflow context	\$execution.id, \$execution.mode, \$workflow.id and \$workflow.name are mapped to runtime headers.	PASS
Execution logs	Code node emits a structured console.log line containing execution and result data.	PASS
Successful execution	The supplied audit notes report a successful live run against JSONPlaceholder.	PASS

Overall assessment coverage: 8/8 requirements satisfied.

3. Node Configuration Details

3.1 Prepare Request Context

The Set node creates the request context used by downstream API calls.

```
requestTimestamp = {{ $now.toISO() }}
requestId = req-{{ $execution.id }}-{{ $workflow.id }}
targetUserId = 1
```

3.2 Get User

The first HTTP Request node dynamically builds the user URL from the current item and sends runtime request headers.

```
URL: https://jsonplaceholder.typicode.com/users/{{ $json.targetUserId }}

X-Request-Id: {{ $json.requestId }}
X-Request-Timestamp: {{ $json.requestTimestamp }}
X-Execution-Id: {{ $execution.id }}
X-Workflow-Name: {{ $workflow.name }}
```

3.3 Get User Posts

The second HTTP Request node demonstrates an ancestor reference: instead of relying only on its immediate input, it reaches back to Prepare Request Context.

```
URL: https://jsonplaceholder.typicode.com/posts
Query userId: {{ $('Prepare Request Context').item.json.targetUserId }}

X-Request-Id: {{ $('Prepare Request Context').item.json.requestId }}
X-Chained-Timestamp: {{ $now.toISO() }}
X-Execution-Mode: {{ $execution.mode }}
```

3.4 Build Result & Execution Log

The Code node collects all posts, reads the user from Get User, reads the request context from Prepare Request Context, and creates a structured final result.

```
const posts = $input.all().map(i => i.json);
const user = $('Get User').first().json;
const ctx = $('Prepare Request Context').first().json;
```

The final object includes requestId, requestTimestamp, completedAt, executionId, executionMode, workflowId, workflowName, user details, postCount, and postTitles.

4. Execution Logging & Validation

The final Code node writes a structured execution log using the runtime execution ID, request ID, user name, post count, and completion timestamp.

```
console.log('[EXEC ' + $execution.id + ']' + result.requestId + ' user=' + result.user.name + ' posts=' + result.postCount + ' at ' + result.completedAt);
```

The supplied audit notes report that the workflow was executed live against the real JSONPlaceholder API, returned the target user and 10 posts, and completed successfully. The audit identifies execution 325 as the submission execution to screenshot.

5. Expected Final Result

Field	Purpose / Expected Value
requestId	Runtime request identifier built from execution and workflow IDs.
requestTimestamp	ISO-8601 timestamp generated at request-context creation.
completedAt	ISO-8601 completion timestamp generated in the Code node.
executionId	Current n8n execution ID.
executionMode	Current n8n execution mode.
workflowId	Current workflow ID.
workflowName	Current workflow name.
user	Selected user's id, name and email.
postCount	Number of posts returned by the posts API call; audit notes report 10.
postTitles	Array of returned post titles.

6. Submission Checklist

- ☐ Completed all assessment tasks
- ☒ Multi-node workflow created
- ☒ \$json expressions used
- ☒ Ancestor node references configured
- ☒ Dynamic HTTP headers implemented
- ☒ ISO-8601 timestamps generated
- ☒ Execution/workflow context variables mapped
- ☒ Execution logs implemented
- ☒ Workflow executed successfully
- ☐ Loom/YouTube demo video attached
- ☐ GitHub public repository or exported n8n workflow attached
- ☐ Screenshots of workflow execution and logs attached

7. Submission Notes

For the final LMS submission, include the Loom/YouTube walkthrough, the public GitHub repository or exported n8n workflow, the workflow/source file, and screenshots showing the workflow, dynamic expressions, ancestor references, final output, and execution logs.

Source basis: the provided Topic 5 assessment requirements and the exported n8n workflow configuration supplied with this submission.